

Занятие 0x02

Языки программирования низкого уровня

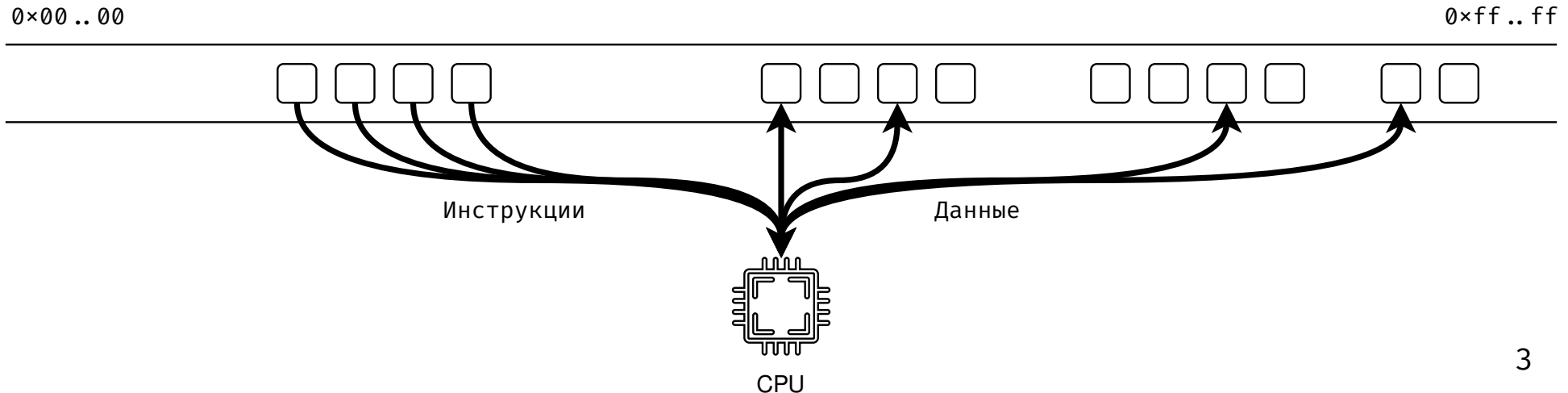
Основы работы с памятью

Устройство ОЗУ

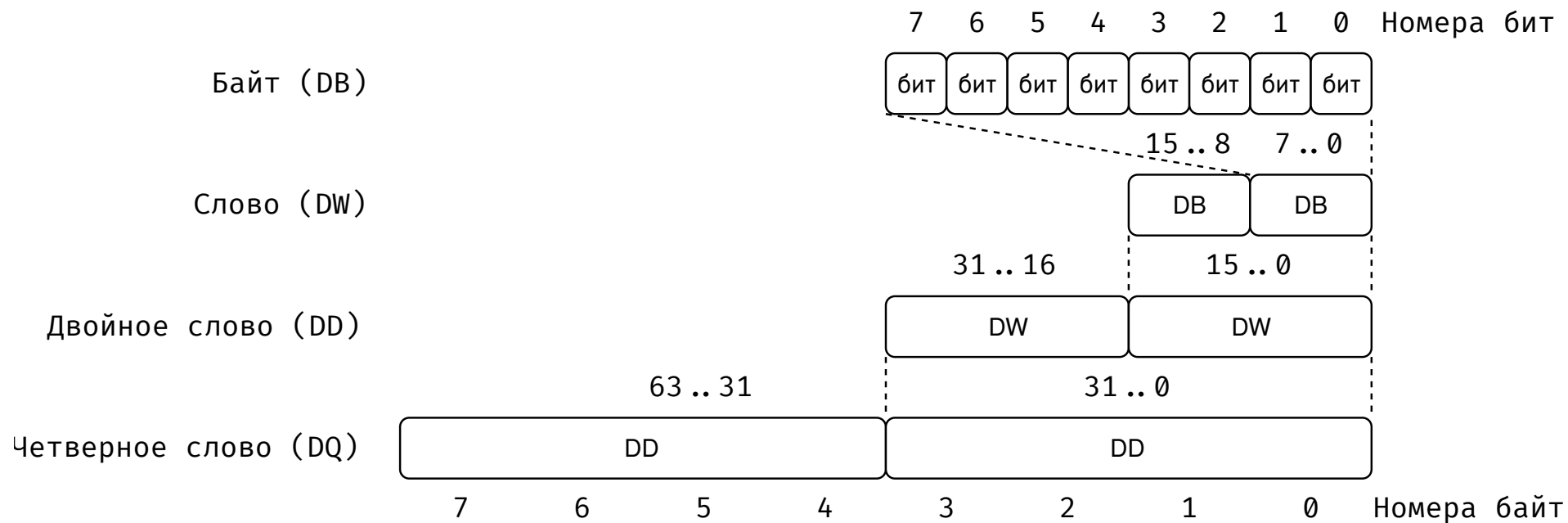
- Нумерованный набор ячеек (байт) для хранения информации
- Каждая ячейка имеет номер — **адрес**
- Доступ только к целой ячейке, а не к отдельным битам

Процессор и память

- И инструкции (код) и данные хранятся в ОЗУ
 - Процессор последовательно считывает инструкции (адрес инструкции в регистре RIP)
 - Если инструкция выполняет обращение к памяти – считывает или записывает значение из памяти (например, инструкция MOV)



Виды данных



Объявление переменных

- Переменная — область памяти для хранения данных
- Необходимо резервировать явно
- Область памяти может иметь метку для удобства обращения к ней

Объявление переменных

<code>var1 db 0x11</code>	<code>; Код обращения к памяти</code>
<code>var2 dw 0x2233</code>	<code>mov al, [var1]</code>
<code>var3 dd 0x44556677</code>	<code>mov bx, [var2]</code>
<code>var4 dq 0x8899aabbccddeeff</code>	<code>mov ecx, [var3]</code>
	<code>mov rdx, [var4]</code>
	<code>mov al, [var4] ???</code>

Двоичное представление

	var1		var2		var3				var4								
00000000	11	33	22	77	66	55	44	ff	ee	dd	cc	bb	aa	99	88	8a	.3"wfUD.....
00000010	04	25	00	00	00	00	66	8b	1c	25	01	00	00	00	8b	0c	.%....f..%....
00000020	25	03	00	00	00	48	8b	14	25	07	00	00	00				%....H..%....
0000002d																	

00000000F	8A0425000000000	mov al,[0x0]
000000016	668B1C2501000000	mov bx,[0x1]
00000001E	8B0C2503000000	mov ecx,[0x3]
000000025	488B142507000000	mov rdx,[0x7]

Адресная арифметика

```
var1 dq 0x1122334455667788    lea rax, [var1]
var2 dq 0x9900aabbccddeeff    mov rbx, [rax + 2]

                                mov rcx, 2
                                mov rdx, [rax + 2 * rcx]

                                lea rsi, [rax + 4 * rcx]
```


Адресная арифметика

- **mov** регистр, [адрес] (или наоборот)
 - Читает значение из памяти по адресу в регистр
- **mov** [адрес], регистр
 - Записывает значение из регистра в память по адресу
- **lea** регистр, [A + B * C + D]
 - Сохраняет в регистр значение адреса, вычисленного по выражению
 - A, B, C, D можно опускать
 - A – регистр
 - B – регистр
 - C – {2, 4, 8}
 - D – число
 - Иногда используют для вычисления выражений

Адресная арифметика

	var1								var2								
00000000	88	77	66	55	44	33	22	11	ff	ee	dd	cc	bb	aa	00	99	.wfUD3".....
00000010	48	8d	04	25	00	00	00	00	48	8b	58	02	b9	02	00	00	H..%....H.X.....
00000020	00	48	8b	14	48	48	8d	34	88								.H..HH.4.

```
00000000  488D042500000000  lea rax,[0x0]
00000008  488B5802           mov rbx,[rax+0x2]
0000000C  B902000000        mov ecx,0x2
00000011  488B1448           mov rdx,[rax+rcx*2]
00000015  488D3488           lea rsi,[rax+rcx*4]
```

Инструкции работы с битами

Инструкция	Описание
<code>shl op1, op2</code>	<code>op1 = op1 << op2</code>
<code>shr op1, op2</code>	<code>op1 = op1 >> op2</code>
<code>and op1, op2</code>	<code>op1 = op1 & op2</code>
<code>or op1, op2</code>	<code>op1 = op1 op2</code>
<code>xor op1, op2</code>	<code>op1 = op1 ^ op2</code>
<code>bts op1, op2</code>	установить бит <code>op2</code> в <code>op1</code>
<code>btr op1, op2</code>	сбросить бит <code>op2</code> в <code>op1</code>
<code>btc op1, op2</code>	инвертировать бит <code>op2</code> в <code>op1</code>
<code>bt op1, op2</code>	проверить бит <code>op2</code> в <code>op1</code>

Пример работы с битами

[bits 64]

```
mov eax, 0xaabbccdd
```

```
shr eax, 12
```

```
and eax, 0xf ; eax = 0xc
```

```
mov eax, 0xaabbccdd
```

```
or  eax, 0x00ff00ff ; eax = 0xaaffccff
```

Символьная информация

- Для кодирования символов используется таблица кодировки
 - Таблица кодировки сопоставляет символы с целыми числами (перенумеровывает)
- Программа работает с кодами символов
 - Для отображения символа передается его номер и «рисуются» нужный символ

Пример работы с символами

Часть таблицы кодировки ASCII

32 пробел	48 0	64 @	80 P	96 `	112 p
33 !	49 1	65 A	81 Q	97 a	113 q
34 "	50 2	66 B	82 R	98 b	114 r
35 #	51 3	67 C	83 S	99 c	115 s
36 \$	52 4	68 D	84 T	100 d	116 t
37 %	53 5	69 E	85 U	101 e	117 u
38 &	54 6	70 F	86 V	102 f	118 v
39 '	55 7	71 G	87 W	103 g	119 w
40 (	56 8	72 H	88 X	104 h	120 x
41)	57 9	73 I	89 Y	105 i	121 y
42 *	58 :	74 J	90 Z	106 j	122 z
43 +	59 ;	75 K	91 [	107 k	123 {
44 ,	60 <	76 L	92 \	108 l	124
45 -	61 =	77 M	93]	109 m	125 }
46 .	62 >	78 N	94 ^	110 n	126 ~
47 /	63 ?	79 O	95 _	111 o	127

[bits 64]

```
mov al, 'a'
```

```
add al, 10
```

```
; в al код символа 'k'
```

```
; (10 символ от 'a')
```

```
00000000 B061      mov al,0x61
00000002 040A      add al,0xa
```